FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	*Benedict Carl R. Roxas*
Section:	*AN42*
Professor:	*Ms. Crisola G. Tan*
Date:	*Oct 5, 2026*

I. BACKGROUND INFORMATION

A **random walk** *is a mathematical process that describes a path consisting of a sequence of* **random steps**. *Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.*

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, *a Python library for* **agent-based modeling**. *Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:*

- *How* **agents move and interact** *in a grid-based environment.*
- *The concept of* **bounded movement**, *ensuring agents stay within the grid limits.*
- *What changing* **parameters** *like the number of agents, grid size, or number of steps affects the system.*
- *How* **agent-based models** *can be used to simulate real-world phenomena, such as* **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. *To understand the concept of* **random walks** *in a grid environment.*
2. *To implement a* **multi-agent simulation** *using Python and AgentPy.*
3. *To visualize the* **movement of agents** *on a grid over time.*

Materials Needed

- *Computer with Python 3.x installed*
- *Libraries: agentpy, matplotlib*
- *Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)*

General Instructions

1. *This* **Technical Assessment (TA4)** *must be completed individually.*
2. *Follow the Task below.*
3. *Include clear comments in your code to describe its logic.*
4. *Each program should display the correct input, process, and output.*
5. *Ensure that your script displays clear and readable outputs in the Command Window.*
6. *Compile all Python codes and corresponding outputs into one PDF file.*
7. *Save it as* **TA1_lastname.pdf**
   (Example: TA1_Patilano.pdf)
8. *Submit the* **python file** *and* **PDF** *file.*

Activity Instructions

Step 1: Run the Code

1. *Open the provided Python code.*

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}

# -------- Run Model -----------
model = RandomWalkModel(parameters)
model.setup()
```

```
# -------- Interactive Animation ----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. *Run the program in your terminal.*
3. *Enter the following values when prompted:*
    - **Number of agents** *(e.g., 5)*
    - **Grid size** *(e.g., 10 for a 10x10 grid)*
    - **Number of steps** *(e.g., 20)*

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. *Each agent is represented by a dot in the grid.*
2. *Agents move* **randomly** *in one of four directions: up, down, left, or right.*
3. *Agents* **cannot leave the grid***; movement is restricted within the grid boundaries.*

Step 3: Observe the Simulation

1. *After running, a* **grid with moving agents** *is displayed.*
2. *Each time step updates the position of all agents.*
3. *Observe:*
    - *How agents spread across the grid over time*
    - *Whether agents cluster in certain areas or move freely*

Step 4: Experiment with Parameters

- *Repeat the simulation with* **different values***:*
    - **More agents** *(e.g., 10 or 20)*
    - **Smaller/larger grid sizes**
    - **Longer simulations** *(e.g., 50 steps)*
- *Observe and record how the* **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths***: Modify the code to* **record and display the path of each agent** *during the simulation.*
2. **Change movement rules***: Make agents* **avoid each other** *or move in a* **preferred direction** *and observe the resulting patterns.*
3. **Analyze final positions***: Record the* **final positions** *of all agents and analyze their* **distribution***.*
4. **Compare scenarios***: Run* **two simulations with different parameters** *and compare the movement patterns.*

Modified (Updated Code - Laboratory Tasks) ⇩

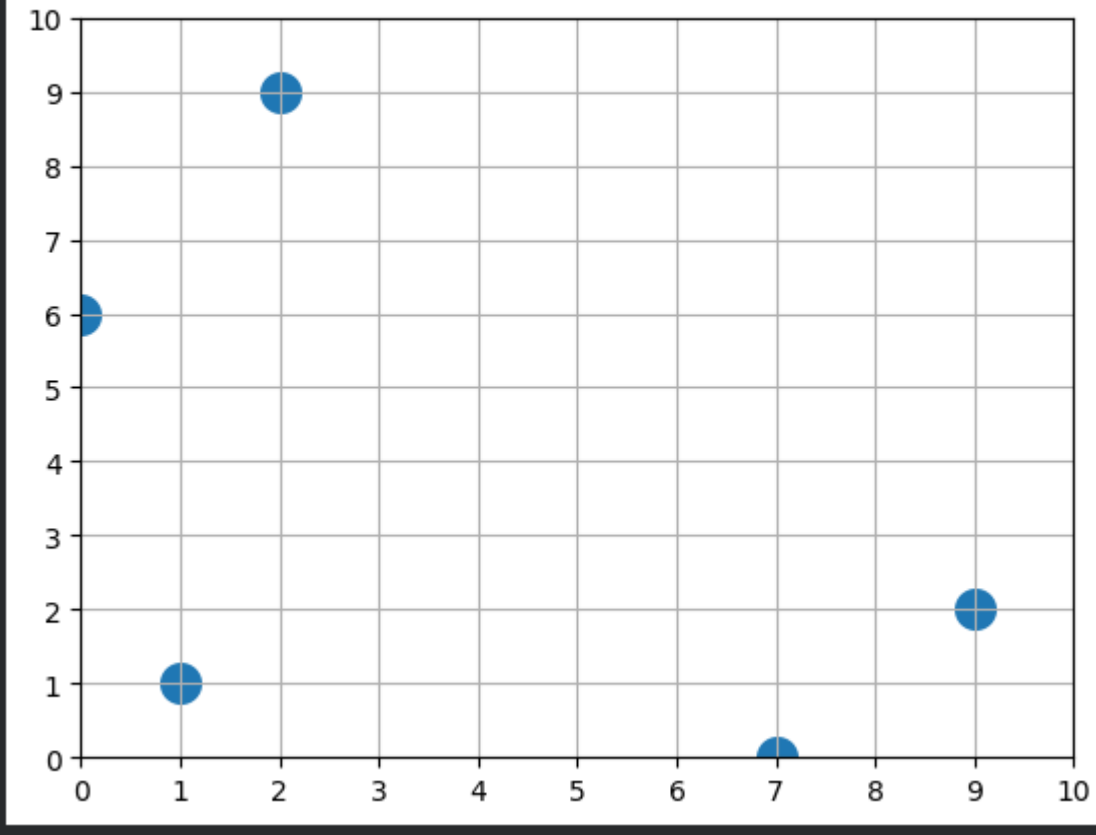
Step 5: Answer the Questions

1. *What happens if the number of agents is* **very high** *compared to the grid size?*
2. *How does increasing the number of steps affect the* **spread of agents***?*
3. *Do the agents ever move outside the grid? Why or why not?*
4. *How can this simulation be used to* **model real-life phenomena** *(e.g., diffusion, crowd movement)?*
5.

Answer the Questions ⇩

1. **If the number of agents is very high compared to the grid size, the agents will become more crowded and may occupy nearby areas.**
2. **Increasing the number of steps gives the agents more time to move around the grid, allowing them to explore and spread to more areas.**
3. **No, the agents do not move outside the grid because their movement is limited by the grid boundaries.**
4. **This simulation can be used to model real-life situations where objects or people move randomly, such as particle diffusion, crowd movement, and animal movement.**

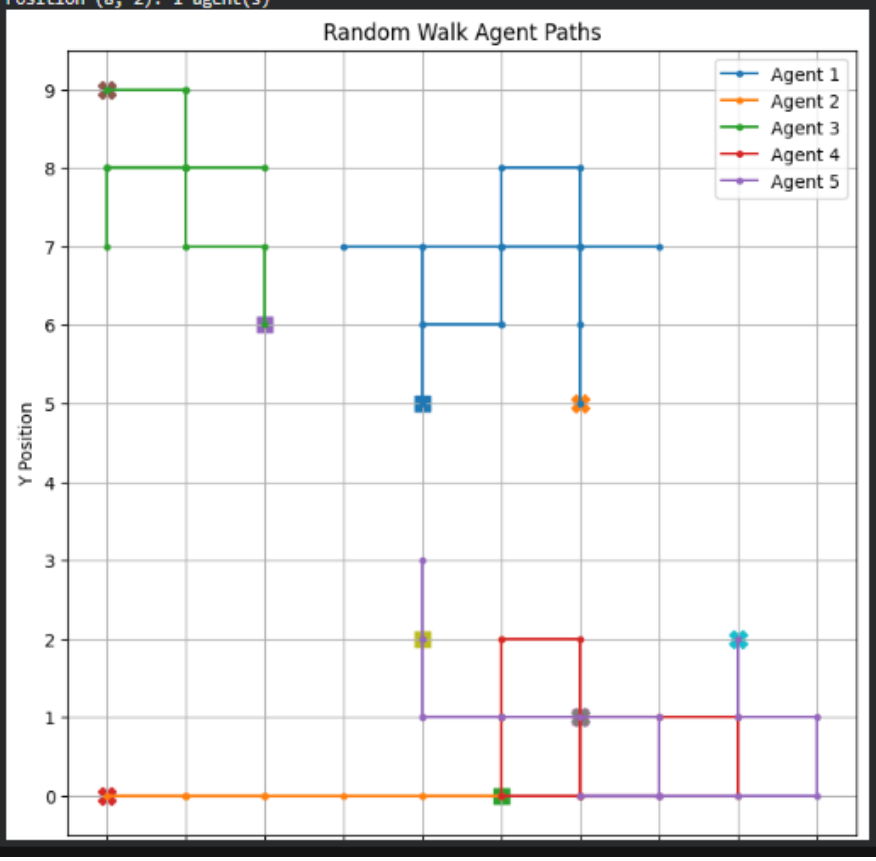
```
Enter number of agents: 5
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 20
```



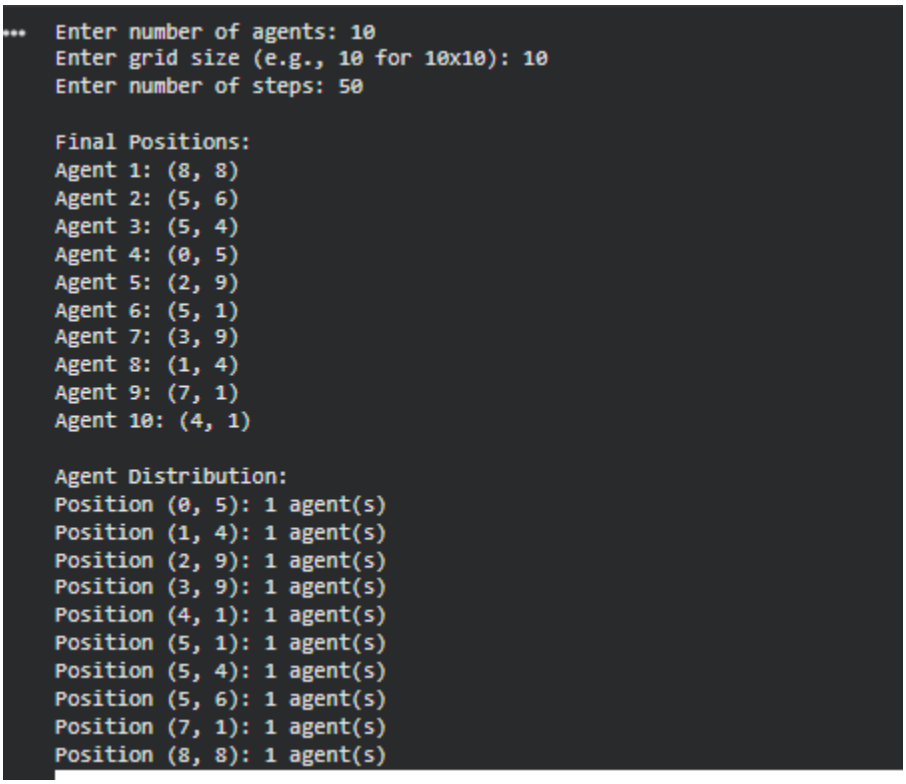
```
Enter number of agents: 5
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 20

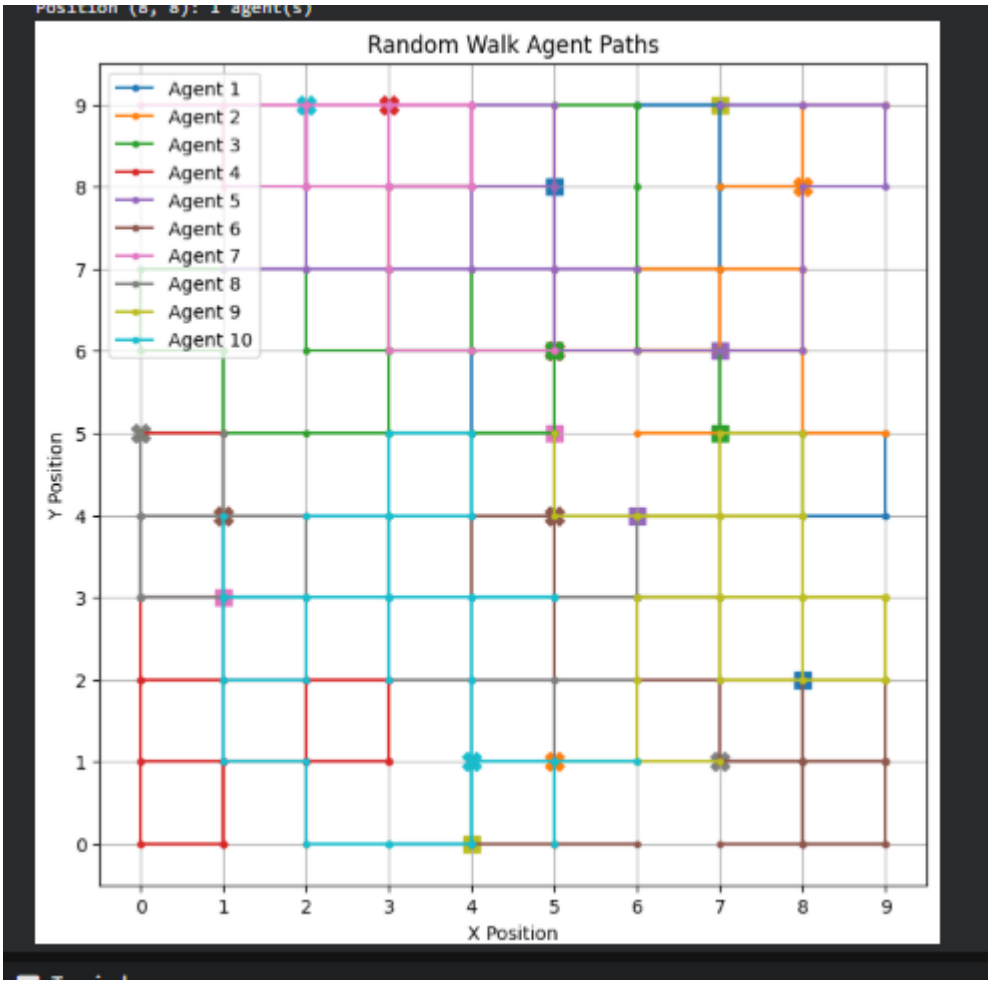
Final Positions:
Agent 1: (6, 5)
Agent 2: (0, 0)
Agent 3: (0, 9)
Agent 4: (6, 1)
Agent 5: (8, 2)

Agent Distribution:
Position (0, 0): 1 agent(s)
Position (0, 9): 1 agent(s)
Position (6, 1): 1 agent(s)
Position (6, 5): 1 agent(s)
Position (8, 2): 1 agent(s)
```



Task 4:


Enter number of agents: 10
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 50
Final Positions:
Agent 1: (8, 8)
Agent 2: (5, 6)
Agent 3: (5, 4)
Agent 4: (0, 5)
Agent 5: (2, 9)
Agent 6: (5, 1)
Agent 7: (3, 9)
Agent 8: (1, 4)
Agent 9: (7, 1)
Agent 10: (4, 1)
Agent Distribution:
Position (0, 5): 1 agent(s)
Position (1, 4): 1 agent(s)
Position (2, 9): 1 agent(s)
Position (3, 9): 1 agent(s)
Position (4, 1): 1 agent(s)
Position (5, 1): 1 agent(s)
Position (5, 4): 1 agent(s)
Position (5, 6): 1 agent(s)
Position (7, 1): 1 agent(s)
Position (8, 8): 1 agent(s)


Random Walk Agent Paths
Agent 1
Agent 2
Agent 3
Agent 4
Agent 5
Agent 6
Agent 7
Agent 8
Agent 9
Agent 10
Y Position
X Position
0
1
2
3
4
5
6
7
8
9